MyScoutee Components

REUSABLE ANGULAR UI

CATALOGUE AND INTEGRATION

Components Guide

Forms, lists, menus, overlays and media controls, with application-owned data and services.

DOCUMENT ID	DOCUMENT VERSION
MSC-COMP-001	1.0.2
APPLIES TO	**RELEASED**
Components 1.4.0	2026-10-12

Document control

This is the canonical HTML source for the Components Guide. The PDF retains a stable filename; Git history records revisions. The catalogue and integration examples were moved from the repository README without changing the library.

Contracts are linked to their public source owners. The consumer supplies its theme, translation catalogue, application data and backend integration.

Contents

Requirements and installation

Component catalogue

Supporting public APIs

Application setup

Styles

Build independently

Develop as a Git submodule

Releases

License — free to reuse with required credit

Make it your own

Contributing

Related project

MyScoutee Components

Reusable Angular UI components extracted from MyScoutee: configurable forms, menus, overlays, lists, calendars, cards and media controls.

Package: `@fssrepository/myscoutee-components` · **Version:** `1.4.0` · **Registry:** GitHub Packages

Component catalogue · Setup · Submodule development · Release notes · License

Requirements and installation

Dependency	Supported peer range
Angular common/core/forms/router	`^21.2.0`
Angular CDK and Material	`^21.2.0`
RxJS	`^7.8.0`
Build/test toolchain	Node.js 24 and npm 11

Configure the package scope in the consuming project's `.npmrc`:

```
@fssrepository:registry=https://npm.pkg.github.com
```

Authenticate using your GitHub identity and a token with `read:packages`, following GitHub's npm registry instructions. Keep the token in your user-level npm configuration or CI secret, never in the repository. GitHub's npm registry requires authentication even for public packages.

```
npm install @fssrepository/myscoutee-components@1.4.0
```

Install compatible peer dependencies in the Angular application. Supply an Angular Material theme and the Material Icons font used by `mat-icon`; the library does not bundle your application's fonts, logos, themes or translation catalogue.

Component catalogue

All components below are exported from `@fssrepository/myscoutee-components`. Import the standalone components you use into the consuming component's `imports`. Source links include the actual input/output contracts; supporting models and config types are exported through the same public API. No internal source path is needed.

Menus, overlays and feedback

Component	Selector	Purpose
`AppMenuComponent`	`app-menu`	Inline, dropdown and sheet menus; actions, radio/checkbox selection, toggles, nested items and drag handling. On/off styling uses the existing toggle kind.
`AppMenuTriggerComponent`	`app-menu-trigger`	Standalone menu trigger presentation for configured labels, icons and state.

Component	Selector	Purpose
`AppMenuOutletComponent`	`app-menu-outlet`	Menu panel/outlet rendering from the shared menu model.
`RateComponent`	`app-menu-rate`	Rating interaction inside menu items.
`PopupComponent`	`app-popup`	Configurable modal surface with header/actions, nested overlay coordination and close handling.
`DialogComponent`	`app-dialog`	Confirmation and input dialogs driven by DialogStore.
`AccordionComponent`	`app-accordion`	Expandable sections with configurable headings and menu actions.
`FloatingLauncherComponent`	`app-floating-launcher`	Floating, draggable action launcher coordinated with other launchers.
`CopyLinkComponent`	`app-copy-link`	Copy-link action using the shared menu presentation.
`PromptComponent`	`app-prompt`	Inline prompt with optional icon, action, busy state and dismissal.
`IndicatorComponent`	`app-indicator`	Loading/progress feedback with timing and visibility handling.

Forms and inputs

Component	Selector	Purpose
`FormFlowComponent`	`app-form-flow`	Model-driven multi-step forms, validation, summaries, header toggles and lazy custom controls.
`DateInputComponent`	`app-date-input`	Single date, date range and time editing; configurable precision, bounds and optional horoscope metadata.
`LinkInputComponent`	`app-link-input`	Editable link value with configured label and length constraints.
`LocationInputComponent`	`app-location-input`	Location value and map actions, with configurable route stops.
`RouteInputComponent`	`app-route-input`	Ordered stop list, lazy route editor, stop reordering and map actions; string-array form value.
`SlotsInputComponent`	`app-slots-input`	Recurring base schedules and date overrides; also reuses the editor for a single recurring task.
`TextCardsInputComponent`	`app-text-cards-input`	Configurable text-card collection and editing actions.

Lists, calendars and cards

Component	Selector	Purpose
`SmartListComponent`	`app-smart-list`	Model-driven list/grid, calendar and timeline rendering, pagination, sorting, filters and item actions.
`CalendarCardComponent`	`app-calendar-card, app-smart-list-page-card`	Calendar/page-card surface driven by the caller’s query, pages and callbacks.
`HeaderCardComponent`	`app-header-card`	Header and identity presentation with configured actions.

Component	Selector	Purpose
`ImageCardComponent`	`app-image-card`	Image-led card content and actions.
`InfoCardComponent`	`app-info-card`	Structured information card with detail and menu interactions.
`PairCardComponent`	`app-pair-card`	Paired card presentation.
`PaymentCardComponent`	`app-payment-card`	Payment information presentation; payment processing remains in the consumer.
`SingleCardComponent`	`app-single-card`	Single-item card presentation.
`SingleRowComponent`	`single-row`	Compact row presentation and actions.
`TextCardComponent`	`app-text-card`	Text-led card with optional menus, counters and status.
`WarpImageCardComponent`	`app-warp-image-card`	Image card with transformed/warp presentation.

Media and branding

Component	Selector	Purpose
`ImageCarouselComponent`	`app-image-carousel`	Form-bound image collection with selection, edit/remove actions and read-only presentation.
`ImageGalleryComponent`	`app-image-gallery`	Gallery entry point for shared image selection and editing state.
`ImageGalleryPopupComponent`	`app-image-gallery-popup`	Gallery popup rendering and image-edit interactions.
`DeploymentBrandComponent`	`app-deployment-brand`	Configurable product name/logo; branding supplied by input or provider.

Supporting public APIs

Area	Exports	Responsibility
Translation	`UiTranslations`, `UI_TRANSLATIONS`, `provideUiTranslations`, `I18nPipe`	Connect the consumer’s existing translation engine and revision signal.
Guides	`ExplanationGuideService`, `EXPLANATION_GUIDE_OPTIONS`, guide types	Guide state and progression; the consumer supplies content, eligible surfaces and persistence scope.
Overlays	`DialogStore`, `PopupPresenceStore`, `OverlayNavigationStore`, `FloatingLauncherRegistry`	Dialog requests, modal presence, nested navigation and launcher coordination.
Media	`ImageGalleryStore`, `UI_IMAGE_UPLOAD`, `UI_IMAGE_DETAILS`, `UI_IMAGE_URLS`	Gallery state and consumer-supplied upload, metadata and URL behavior.

Area	Exports	Responsibility
Form extension	`FORM_FLOW_CONTROLS`, `FormFlowControls`, `FormFlowControlDirective`	Register actual controls lazily through their existing value accessor. The route renderer is built in.
Calendar conversion	`CalendarCardConverter`, `calendarCardConverter`	Convert a caller-owned calendar model while retaining pages, query and callbacks.
Schedules	`SlotTemplate`, `normalizeSlotTemplates`, `normalizeSlotOverrideDate`, `normalizeScheduleFrequency`	Normalize shared scheduling fields while preserving consumer metadata.
Lists	Query/page models, finite/infinite steppers, calendar/timeline/page adapters	Data-source-neutral list rendering and navigation.
Dates and presentation	`UiDateUtils`, `AppCalendarDateAdapter`, `AppCalendarDateFormats`, number/rating helpers, horoscope data	Shared date, numeric and presentation operations.
Rendering	`ViewportSeenDirective`, `LazyBgImageDirective`, `CounterBadgePipe`, schedulers	Viewport awareness, deferred image rendering, counters and scheduled UI work.

The implementation of each public export is indexed in src/public-api.ts. The route popup is an internal lazy implementation owned by RouteInputComponent.

Application setup

The library calls your existing adapters; it contains no MyScoutee backend, Firebase setup, domain repositories or application session.

Provider/configuration	When to supply it
`UI_TRANSLATIONS` or `provideUiTranslations(YourI18nService)`	Components that render translated labels. The contract includes `revision`, `currentLanguage`, `translate` and `translateParams`.
`EXPLANATION_GUIDE_OPTIONS`	Guide-aware controls and overlays; configure context eligibility and `loadState`. Use an empty state when the application does not use guides.
Angular Router providers	Overlay navigation, such as menus and popups; reuse the application's existing router.
`UI_BRANDING` or the branding input	DeploymentBrandComponent. The token provides a function returning the current branding model.
`UI_IMAGE_UPLOAD`	Image uploading; provide the consumer's upload implementation.
`UI_IMAGE_DETAILS`	Additional image fields or domain-specific validation.
`UI_IMAGE_URLS`	URL variants, seeded-image conventions or a card fallback image. Default URLs pass through unchanged.
`UI_LIST_DEFAULTS`	Override the shared loading-window default.
`FORM_FLOW_CONTROLS`	Add or override form renderers; providers share a scope within one form.

A minimal translation/guide configuration for an application without a translation catalogue or guide content looks like this:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import {
  UI_TRANSLATIONS, EXPLANATION_GUIDE_OPTIONS, type UiTranslations
} from '@fssrepository/myscoutee-components';

const translations: UiTranslations = {
  revision: () => 0,
  currentLanguage: () => 'en',
  translate: (value, fallback) => fallback ?? value ?? '',
  translateParams: (value, params, fallback) =>
    Object.entries(params).reduce(
      (text, [key, replacement]) => text.replaceAll(`{${key}}`, String(replacement)),
      fallback ?? value ?? ''
    )
};

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter([]), // Use the consuming application's routes.
    { provide: UI_TRANSLATIONS, useValue: translations },
    { provide: EXPLANATION_GUIDE_OPTIONS, useValue: {
      isExplainableContext: () => false,
      loadState: async () => ({ activeRevision: null, guideFields: [] })
    } }
  ]
};
```

For an existing application, bind its real translation and guide services instead. A standalone date field can then use the standard Angular value accessor:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { DateInputComponent, type DateInputValue } from '@fssrepository/myscoutee-components';

@Component({
  selector: 'example-date-field',
  imports: [FormsModule, DateInputComponent],
  template: `<app-date-input [(ngModel)]="date"
    [model]="{ mode: 'single', precision: 'date', valueFormat: 'iso-date',
               field: { label: 'Date' } }" />`
})
export class ExampleDateField {
  date: DateInputValue = '2026-10-10';
}
```

Menus, routes and schedules

- Use an AppMenu item with `kind: 'toggle'` and `toggleStyle: 'on-off'` for the on/off presentation. Existing menu events, disabled handling and form-bound selected-value arrays remain the value contract. FormFlow header toggles keep their boolean binding; no wrapper component is needed.
- RouteInput uses a `string[]` value. Its popup edits a draft and commits on save. Asset assignment rules and translated domain labels belong to the consumer.

- SlotsInput handles common schedule fields. Use `scheduleOnly` for a recurring task without slot rows. Supply `labels`, `frequencyLabel` and, when needed, `normalizeTemplates` through SlotsInputConfig. Event subdefinitions and other domain metadata remain consumer-owned.

Styles

Import shared Sass through the public package paths:

```
@use '@fssrepository/myscoutee-components/styles' as ui;
@use '@fssrepository/myscoutee-components/styles/contextual-colors' as colors;
@use '@fssrepository/myscoutee-components/styles/popup-close-button' as popupClose;
```

The root module forwards popup, motion, Material-control, form-control and SmartList-shell mixins. Additional public modules are `contextual-colors`, `media-badge`, `popup-close-button` and `rate-card.shared`. The consuming application owns global layout, theme variables and the Material theme.

Build independently

```
npm ci
npm run build
npm test
```

`npm run watch` rebuilds this library when its source changes. The build requires only this repository and its declared dependencies. Generated `dist/`, caches and `node_modules/` are ignored. Component/store/helper tests live beside their owners. The root manifest is private to prevent accidentally publishing workspace sources; `npm run prepare:release` prepares the built `dist/` manifest for publication.

Develop as a Git submodule

The source repository is `myscoutee-components`; the consuming directory is `components`:

```
git submodule add -b master https://github.com/fssrepository/myscoutee-components.git components
```

In an existing checkout:

```
git submodule update --init --recursive --merge components
git -C components switch master
npm ci
```

The consuming package declares `"workspaces": ["components"]` and a matching `@fssrepository/myscoutee-components` dependency. MyScoutee's build/serve/test entry points build this workspace before the application; its dev stack uses the existing `frontend-sync` process. Do not run a competing build alongside that process.

When editing a separate checkout, commit there and fast-forward the consuming submodule on `master`. Keep both checkouts on `master`. The application consumes the same implementation through the public package API; the monolith remains supported.

Releases

CHANGELOG.md and releases/ record versioned changes. Releases are built and published locally. After a completed build:

```
npm run prepare:release
npm pack ./dist --ignore-scripts
npm publish ./fssrepository-myscoutee-components-1.4.0.tgz --registry=https://npm.pkg.github.com
```

The preparation script fixes package-relative entry points, includes the license and documentation, and excludes source maps, tests and build caches from the archive. If `frontend-sync` is already running, use a separate copy of its completed library output in the release checkout's ignored `dist/`; do not run a competing build or change the output being watched by the application. Preparation replaces ng-packagr's watch timestamp version with the source manifest's release version.

Tag the verified source with `vX.Y.Z`, push the tag and create a GitHub Release using its versioned release notes. Attach the published `.tgz` and its SHA-256 checksum. GitHub Packages publication requires a local login with `write:packages` permission. No registry token is stored in source and no GitHub Action is required.

License — free to reuse with required credit

The repository's original code and documentation are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Anyone may copy, modify and redistribute them, including commercially.

Attribution is a condition of the license, not an optional request. When sharing the material, including modified versions, follow **Section 3(a)** of LICENSE: credit **MyScoutee (fssrepository)**, retain the required notices, link to the source and license, and indicate changes.

Place the credit somewhere readers or users can access it, as appropriate to what you share: a README, documentation, an About/Credits page or a website footer. The full, unmodified standard license text is in LICENSE. The attribution information for this repository is supplied in the credit below. This requirement applies when you share the licensed material; private use alone does not require publishing a credit.

Example credit (also describe changes if you made any):

Uses MyScoutee Components by MyScoutee (fssrepository). Copyright © 2026, fssrepository. Licensed under CC BY 4.0. Provided without warranties; see LICENSE, Section 5.

MyScoutee logos and brand artwork are excluded from CC BY 4.0 and follow BRANDING.md. Separately identified third-party material retains its own license and notices.

Make it your own

Use your own project name and logo. Any MyScoutee logo in template source, examples or UI is demonstration branding: remove or replace it wherever it identifies your product, including an adapted README header.

You may display the MyScoutee logo in a clearly labeled source-credit section or when demonstrating the original MyScoutee project. It must not suggest that your project is MyScoutee, an official MyScoutee service, or endorsed by its maintainers. A text credit satisfies attribution; displaying the logo is not required. See BRANDING.md.

Contributing

Bug reports, documentation improvements and focused pull requests are welcome. Read CONTRIBUTING.md to get started.

Related project

MyScoutee Template is the companion repository for reusable project infrastructure.